FX

제안을 코드 효과로 바꾸는 런타임

Tool 계약, FileEdit의 동시성, Bash 보안, permission과 partial effect

모델은 효과를 제안하지만, 실행 가능성과 안전성은 도구 런타임이 입력별로 다시 결정한다.

대상
v2.1.88 / 2ca5ddabfed5

관점
effect · authority · repository change

제3자 복원 소스에 대한 교육용 분석이다. 누락된 모듈과 복원 한계를 구현 사실로 보완하지 않는다.

Tool은 함수가 아니라 효과 계약이다

Tool 하나는 모델에게 보이는 행동 언어, runtime validation, permission, 외부 효과, 결과 protocol, 사용자 표시를 함께 규정한다.

01	언어	name·description·prompt·schema가 모델이 생성할 행동을 정의
02	정책	isReadOnly·isDestructive·permission matcher가 authority를 제한
03	실행	call이 filesystem/process/network 효과를 발생
04	관찰	progress와 result가 model/UI/SDK에 서로 다른 표현으로 환류
05	운영	telemetry·result budget·compatibility alias가 배포 환경을 지지

왜 핵심인가	LLM의 자유로운 출력과 운영체제의 비가역 효과 사이에서 의미와 권한을 번역하는 경계이기 때문이다.

하나의 Tool 타입에 모인 다섯 변화 이유

현재 Tool 계약은 응집력이 높은 protocol seam인 동시에 변경 결합의 중심이다.

facet	대표 멤버	함께 바뀌는 이유
model contract	inputSchema, prompt, description, strict	모델이 올바른 호출을 만들게 함
effect semantics	call, isReadOnly, isConcurrencySafe, interruptBehavior	실행 순서와 취소 결정
authority	validateInput, checkPermissions, preparePermissionMatcher	실행 승인과 값 수정
protocol mapping	mapToolResultToToolResultBlockParam	다음 reasoning에 결과 전달
presentation	renderToolUse/Result/Progress/Error	interactive projection

이 통합은 새 도구를 만들 때 빠뜨릴 항목을 줄인다. 반면 headless core가 React type을 알게 하고, 작은 UI 변경도 Tool 모듈 전체에 영향을 준다. 리팩터링은 장단점을 모두 보존해야 한다.

코드 좌표 src/Tool.ts

buildTool은 기본값을 중앙화하지만 의미까지 안전하게 만들지는 않는다

generic builder는 부분 정의를 완전한 Tool로 바꾸며 literal name과 메서드 타입을 보존한다.

type-level builder의 역할

```
ToolDef<Input, Output>
  + centralized defaults
  → Tool<Input, Output>

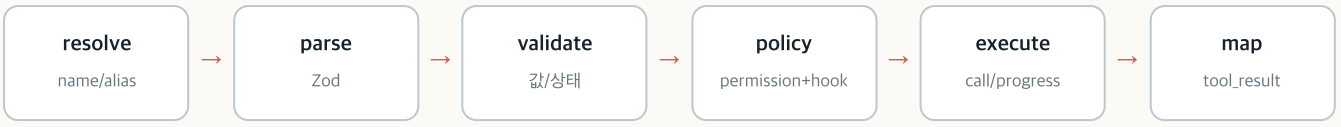
compile-time: required shape, inferred input/output
runtime: schema parse, policy, effect
```

`isConcurrencySafe=false`와 `isReadOnly=false` 기본은 보수적이다. 그러나 `isDestructive=false`는 실제 비가역성을 자동 판정하지 않는다. 안전성은 builder 한 곳이 아니라 general permission과 tool-specific validation의 합성 결과다.

코드 좌표 src/Tool.ts

tool_use는 여섯 단계에서 신뢰를 얻는다

dispatch는 tool 이름만 보고 call하지 않는다. 각 단계는 서로 다른 종류의 오류를 model-visible result로 바꾼다.



실패	표현	다음 행동
unknown tool	is_error tool_result	모델이 다른 도구 선택
schema mismatch	구조화 validation error	입력 수정/ToolSearch
domain invalid	tool-specific errorCode/message	read·path·명령 수정
permission deny	denial + optional updated input	사용자 선택 반영
call exception	sanitized tool error	후속 reasoning 또는 종료

코드 좌표 src/services/tools/toolExecution.ts

Zod는 TypeScript가 닿지 못하는 경계를 닫는다

모델이 만든 JSON, MCP payload, resume data는 compile-time 타입의 보호를 받지 않는다. schema parse 뒤에야 typed input으로 취급할 수 있다.

```
unknown JSON —safeParse→ validated value —domain checks→ admissible effect
```

schema와 validateInput은 중복이 아니다. schema는 *모양*을, validateInput은 현재 filesystem·permission·read state와 관련된 *의미*를 검사한다. FileEdit에서 문자열이 모두 존재해도, 파일을 읽지 않았거나 old_string이 여러 곳에 있으면 허용되지 않는다.

코드 좌표 src/services/tools/toolExecution.ts · src/tools/FileEditTool/FileEditTool.ts

Read는 무해한 함수가 아니다

파일 읽기는 비밀·device path·대용량·이미지/PDF·working-directory permission을 다루며, 이후 Edit 권한을 만드는 관찰이기도 하다.

책임	구현 노력	왜 필요한가
path admissibility	blocked device/permission/alternate path	credential leak·hang·scope 이탈 방지
content limits	line/token/image budget	OOM과 context 폭발 방지
format	line number·binary/media 변환	모델이 후속 edit를 정확히 제안
freshness	content/timestamp/partial state 기록	optimistic edit validation
projection	model result와 UI 표시 분리	정확한 context와 읽기 쉬운 terminal

> **설계 의미**
> Read는 “정보를 반환하는 도구”이면서 “이 파일에 대해 어느 시점의 관찰을 가졌는가”를 발급하는 capability다.

코드 좌표 src/tools/FileReadTool/FileReadTool.ts · src/tools/FileReadTool/FileReadTool.ts

Edit는 old_string 치환보다 검증이 더 크다

파일을 쓰기 전에 path, secret, deny rule, 크기, encoding, 존재, notebook, read state, mtime, match cardinality, settings validity를 확인한다.

01	scope	정규화된 경로와 deny rule, UNC 부작용
02	resource	파일 크기·encoding·존재 여부
03	knowledge	full read가 있었고 partial view가 아님
04	freshness	read 뒤 사용자/linter 변경이 없음
05	intent	old_string이 존재하고 replace_all 의미가 명확
06	domain	settings/notebook 같은 특수 파일 규칙

이 계층은 모델의 잘못된 제안을 사용자 데이터 손실로 바꾸지 않는 방어선이다. errorCode와 구체적 메시지는 단순 UX가 아니라 모델이 다음 행동을 수정할 feedback protocol이다.

코드 좌표 src/tools/FileEditTool/FileEditTool.ts

read-before-write는 optimistic concurrency control이다

Edit는 파일을 lock한 채 모델이 생각하도록 만들지 않는다. 먼저 읽고, 쓰기 직전에 여전히 같은 상태인지 확인한다.

FileEdit의 낙관적 동시성 모델

```
read:
  state[path] = { content, timestamp, fullView }

edit preflight:
  require fullView
  if mtime advanced and content differs: reject

critical section:
  sync read → compare → patch → sync write
```

mtime만 믿지 않고 Windows cloud sync/antivirus의 false positive를 content 비교로 보정한다. 이는 version number 대신 timestamp+snapshot을 쓰는 OCC다.

코드 좌표 src/tools/FileEditTool/FileEditTool.ts · src/tools/FileEditTool/FileEditTool.ts

“await를 두지 않는다”가 동시성 설계다

directory 생성과 history backup은 critical section 밖에서 await한다. staleness check와 write 사이에 yield하면 다른 edit가 끼어들 수 있기 때문이다.

> **핵심 코드 판단**
>
> Node의 single thread가 race를 없애 주지 않는다. `await`는 event loop에 제어를 돌려주므로 check-then-act 사이가 열릴 수 있다. 이 구간은 synchronous read/patch/write로 좁혀져 있다.

다만 이것은 process 내부의 interleaving만 줄인다. 다른 process나 editor가 check 직후 write할 수 있는 TOCTOU 가능성, crash 중간 상태, filesystem atomicity는 별도 문제다.

코드 좌표 src/tools/FileEditTool/FileEditTool.ts · CWE-367 TOCTOU

쓰기 이후의 합류가 변경을 완성한다

파일에 bytes를 쓰는 것만으로 작업이 끝나지 않는다. 관련 projection과 future guard를 함께 갱신해야 한다.



LSP/IDE 알림 실패는 파일 쓰기를 되돌리지 않는다. 즉 이 작업은 ACID transaction보다 saga에 가깝다. 핵심 effect와 보조 effect의 실패 정책을 구분하고, 어느 단계까지 성공했는지 관찰 가능해야 한다.

코드 좌표 src/tools/FileEditTool/FileEditTool.ts

Bash permission은 문자열 prefix 비교로 끝나지 않는다

shell command는 quoting, substitution, redirect, pipeline, wrapper, environment variable을 가진 언어다. 같은 표면 prefix가 전혀 다른 효과를 만들 수 있다.

구문/환경	위험	필요한 해석
operators ; && |	허용 command 뒤에 effect 추가	subcommand 분해와 개별 판정
redirection > >>	read command가 write로 변함	출력 경로 추출
$(...) / backticks	숨은 command 실행	quote/substitution 분석
env/PATH/LD_*	binary hijack·동작 변경	leading env 정규화와 deny
heredoc/escaped newline	parser desync·content 실행	shell grammar에 가까운 검사
wrappers timeout/env/stdbuf	실제 base command 은닉	wrapper stripping

분포가 말하는 것

Bash/PowerShell·parser·security 관련 선택 영역이 약 3.9만 줄인 것은 shell을 범용 effect 통로로 제공하는 비용이 얼마나 큰지 보여 준다.

Bash 실행 전 방어는 여러 층의 합성이다

어떤 한 classifier도 모든 command semantics를 증명하지 못한다.

01	**syntax/security guard**	malformed token, quote, substitution, Unicode, injection 패턴
02	**semantic classification**	read-only·filesystem write·git internal path 등
03	**permission rules**	exact/prefix/wildcard allow·ask·deny
04	**path constraints**	추출된 입력/출력 경로가 허용 scope 안인지 확인
05	**sandbox decision**	가능할 때 OS 격리로 effect 범위 축소
06	**human/classifier**	정적 규칙이 확정하지 못한 호출을 재판정

배경지식

OWASP의 핵심 원칙처럼 가능한 경우 shell 자체를 피하고 parameterized API를 쓰는 것이 강하다. 그러나 코딩 에이전트는 shell이 제품 능력의 핵심이므로, 여기서는 parser·policy·sandbox·prompt를 겹친 defense-in-depth를 택한다.

코드 좌표 src/tools/BashTool/bashSecurity.ts · src/tools/BashTool/bashPermissions.ts · OWASP OS Command Injection Defense

permission mode는 UX 설정이 아니라 effect policy다

default, acceptEdits, dontAsk, plan, bypassPermissions 등은 같은 tool call의 admissibility와 사용자 개입 방식을 바꾼다.

결정	의미	주의
allow	현재 입력으로 실행	rule source와 updatedInput 추적
ask	사람 또는 interactive handler 필요	headless/background에서 prompt 불가
deny	효과 차단	모델에게 수정 가능한 설명 제공
mode restriction	도구 종류/쓰기 자체를 제한	plan mode와 실제 read-only는 동일하지 않음
bypass	대부분의 prompt 생략	가용 조건과 위험 표시가 별도

중요한 것은 permission이 tool name만 보지 않는다는 점이다. `Bash(ls)`와 `Bash(rm)`, `Edit`의 경로가 다르므로 입력별 판정이 필요하다.

코드 좌표 src/types/permissions.ts · src/Tool.ts

병렬화 단위는 도구 종류가 아니라 입력별 효과다

runTools는 isConcurrencySafe(parsedInput)를 호출해 연속된 safe call만 batch로 묶고, 나머지는 직렬화한다.

보수적 scheduling

```
for each tool_use:
  parse input
  safe = parse.ok && tool.isConcurrencySafe(input)
  if classifier throws: safe = false

consecutive safe calls → bounded parallel
effectful calls        → serial
```

contextModifier도 병렬 실행 중 즉시 적용하지 않고 tool_use 순서대로 합류한다. 이는 execution concurrency와 state commit order를 분리한 설계다.

코드 좌표 src/services/tools/toolOrchestration.ts · src/services/tools/toolOrchestration.ts

더 빠른 실행은 중복 효과 위험을 만든다

tool_use block이 스트림에서 완성되는 즉시 실행하면 latency를 줄일 수 있지만, 같은 request가 fallback/retry될 때 동일 효과가 두 번 발생할 수 있다.

전략	장점	새 위험
응답 종료 후 실행	attempt 확정 뒤 effect 발생	tool latency가 직렬로 추가
streaming tool execution	모델 나머지 출력과 tool 실행 중첩	partial attempt fallback의 중복 실행
non-streaming fallback	끊긴 stream 회복	이미 시작한 tool과 새 tool_use 충돌

구현이 보여 주는 trade-off

claude.ts에는 streaming fallback이 active tool 실행과 결합될 때 double execution을 만들 수 있다는 주석과 kill switch가 있다. 성능 최적화는 effect idempotency와 함께 설계되어야 한다.

코드 좌표 src/query.ts · src/services/api/claude.ts

취소는 rollback이 아니라 더 이상의 진행을 막는 신호다

AbortSignal은 model stream과 tool에 전달되지만 이미 쓴 파일, 보낸 요청, 시작한 process가 자동으로 원상복구되지는 않는다.

전파

signal

blocking boundary가 signal을 관찰해야 실제로 멈춘다.

정합성

synthetic result

시작된 tool_use는 취소돼도 matching result가 필요하다.

정리

finally/dispose

stream body, timer, lock, child process를 해제한다.

보상

history/cleanup

가능한 effect만 되돌리거나 다음 turn에 상태를 설명한다.

설계 질문	"abort를 지원하는가?"가 아니라 "각 effect point 전·후 취소 시 무엇이 남고, 어떤 메시지와 durable record가 그 사실을 표현하는가?"를 묻는다.

hook은 확장점이면서 effect pipeline의 일부다

PreToolUse는 실행 전 입력·권한을 바꿀 수 있고 PostToolUse는 관찰·후속 context를 만들 수 있다. 위치에 따라 예방 가능성과 책임이 다르다.

시점	가능한 일	불가능/위험
UserPromptSubmit	입력 block, context 추가	이미 발생한 외부 effect 없음
PreToolUse	deny/ask/update input	core schema·permission과 충돌 가능
PermissionRequest	조직 정책·사람 승인 연결	무응답/timeout 처리 필요
PostToolUse	결과 관찰·annotation	이미 발생한 effect 예방 불가
Stop	continuation block/retry	무한 retry guard 필요

구조 원칙

hook 결과를 authority 자체로 취급하지 않고 core permission decision에 명시적으로 합성해야 한다. 또한 hook progress와 final decision을 같은 stream에 섞을 때 terminal semantics를 분리해야 한다.

MCP는 새 Tool 공급자이지 별도 core가 아니다

외부 server의 tool도 동일한 resolve→schema→permission→call→result 경계를 통과해야 제품 안전성이 유지된다.

MCP가 추가하는 것은 remote transport, server identity, authentication, URL elicitation, metadata, dynamic connection이다. 이것들은 adapter 책임이다. model이 보는 관점에서는 built-in Tool과 유사하지만 runtime은 remote trust와 failure를 더 다뤄야 한다.

경계	추가 질문
discovery	서버가 광고한 schema를 언제/얼마나 prompt에 포함?
identity	tool name prefix와 alias 충돌을 어떻게 방지?
permission	서버 annotation을 신뢰할 것인가, local policy로 재판정할 것인가?
transport	timeout·disconnect·partial result를 어떤 tool_result로 만들 것인가?

코드 좌표 src/services/mcp/client.ts · src/services/tools/toolExecution.ts · MCP Tools specification

Tool을 다섯 interface로 찢기 전에 조합점을 설계한다

현재 계약의 문제는 크기 자체보다 서로 다른 consumer가 한 타입을 공유한다는 것이다.

점진적 분해 후보

```
ToolCapability  { name, schemas, modelPrompt }
ToolEffect      { validate, execute, interrupt }
ToolPolicy      { classify, permissionMatcher }
ToolProjection  { toModelResult, toSDKEvent }
ToolPresenter   { renderUse, renderResult, renderProgress }

ToolDefinition = compose(identity, capability, effect, policy, projections)
```

- identity와 input/output schema는 모든 facet을 묶는 단일 source of truth로 유지한다.
- headless path는 Presenter를 import하지 않아도 되게 한다.
- permission과 concurrency classification이 같은 parsed input을 사용하게 한다.
- compatibility alias와 observer backfill은 adapter로 격리한다.

> **위험**
> 한 번에 모든 tool을 migration하면 schema/prompt/render drift를 만들기 쉽다. 먼저 adapter로 기존 Tool을 새 facet에 투영하고 contract fixture를 만든 뒤 하나씩 옮긴다.

Edit와 Bash를 함께 쓰는 변경의 의미

코드 수정 후 테스트라는 흔한 작업은 두 종류의 effect를 합성한다.

단계	Edit	Bash
사전 지식	full read snapshot 필요	command grammar/working directory 필요
권한	path와 write permission	subcommand·path·sandbox·rule
동시성	write는 직렬	read-only command만 제한적 병렬
실패	stale content·ambiguous match	exit code·timeout·signal·background process
결과	patch와 updated file state	stdout/stderr와 command semantic
보상	file history 가능	외부 effect는 일반적 rollback 불가

학습 결론 도구 구현은 API wrapper가 아니라 effect semantics를 제품 계약으로 만드는 작업이다.

좋은 effect runtime의 조건

기능 수보다 효과의 해석 가능성과 실패 후 정합성이 중요하다.

- 모델 input은 실행 전 runtime schema와 현재 상태 검증을 거친다.
- permission은 tool name이 아니라 구체 입력과 effect를 본다.
- read-only/concurrency-safe/destructive는 서로 다른 속성으로 유지한다.
- 취소·retry·fallback에서 중복 효과와 orphan protocol을 막는다.
- 핵심 effect와 LSP/UI/telemetry 같은 후속 effect의 실패 정책을 구분한다.
- 모든 결과는 다음 reasoning과 durable session이 이해할 수 있는 표현으로 합류한다.